

GoREST API AUTOMATION FRAMEWORK

A CAPSTONE PROJECT REPORT

Submitted By

Anirban Bhattacharya

Enrollment Number: 12023002028041

Registration Number: 304202300900959

CANDIDATE DECLARATION

I hereby declare that the project report entitled "GoREST API Automation Framework" presents the work carried out by me as part of the Wipro-organized Quality Engineering and Automation value-added course.

The implementation, test cases, source code, experimental results, and supporting documentation presented in this report are entirely based on the work completed during the project. All automated procedures and evaluation evidence represent my own work.

I have prepared this report using the project implementation and the available execution evidence, without exaggeration of project capabilities or validation results.



Anirban Bhattacharya

Enrollment Number: 12023002028041

Registration Number: 304202300900959

ACKNOWLEDGEMENT

I would like to express my sincere appreciation to Wipro for organizing the Quality Engineering and Automation value-added course. The practical training sessions, coursework modules, and hands-on exercises provided valuable guidance in understanding test automation frameworks, RESTful API testing, and industry testing practices.

I also acknowledge the developers and contributors of the open-source libraries and tools utilized throughout this work, including Python, Behave, Requests, Urllib3, Faker, JSONSchema, and Allure. These open-source technologies provided a reliable and extensible foundation for designing, executing, and reporting the automated test suite.

Finally, I express my heartfelt gratitude to my family and peers for their continuous support, encouragement, and patience throughout the completion of this project.

Anirban Bhattacharya

Anirban Bhattacharya

Enrollment Number: 12023002028041

Registration Number: 304202300900959

TABLE OF CONTENTS

ABSTRACT.....	1
CHAPTER – 1: INTRODUCTION.....	3–4
CHAPTER – 2: LITERATURE SURVEY.....	5–7
CHAPTER – 3: PROBLEM STATEMENT	8–9
CHAPTER – 4: PROPOSED SOLUTION.....	10–14
CHAPTER – 5: EXPERIMENTAL SETUP AND RESULT ANALYSIS	15–20
CHAPTER – 6: CONCLUSION & FUTURE SCOPE	21–22
APPENDIX.....	23–24
BIBLIOGRAPHY	25

ABSTRACT

Automating test procedures for RESTful Application Programming Interfaces (APIs) is an essential activity in contemporary software quality engineering. Cloud-hosted microservices and public web APIs present several practical challenges during automated test execution, including transient network latency, server-side rate limiting, database record collisions on repeated test runs, accidental credential leakage in automated logs, and silent breaking schema modifications that bypass conventional HTTP status code assertions. This capstone project addresses these challenges through the design and implementation of a structured, resilient Behavior-Driven Development (BDD) test automation framework developed specifically for the live GoREST RESTful API service.

The framework is implemented in Python 3.13.2 utilizing the Behave BDD engine, enabling test scenarios to be written in standard human-readable Gherkin syntax (Given-When-Then). The technical architecture incorporates several modular quality engineering components: an HTTP client equipped with urllib3 Retry logic and exponential backoff to handle transient 429 and 5xx responses; a dynamic test data generation engine using Faker to create unique user payloads and minimize email collision errors; rigorous JSON Schema contract validation adhering to Draft 2020-12 specifications to verify payload structure and field data types; an automated teardown mechanism that tracks created entities for teardown at the conclusion of test scenarios; and a security redaction filter that masks Bearer authentication tokens in the relevant test logs and Allure attachments.

The automation suite was evaluated against the live GoREST users endpoint across CRUD lifecycle coverage, with the framework automating user lifecycle operations including POST, GET, and PATCH, with DELETE implemented as automated post-scenario teardown, alongside negative validation cases and security boundaries. In the primary functional suite, all five planned scenarios passed successfully, achieving a 100% functional pass rate across 23 individual test steps, validating user creation (HTTP 201), retrieval (HTTP 200), partial field updates (HTTP 200), unauthorized access rejection (HTTP 401), and non-existent resource handling (HTTP 404). In a separate boundary verification, an observed response time of 2.049 seconds exceeded the configured 2.0-second response-time threshold, causing the SLA assertion to fail as expected and confirming that the framework correctly detects response-time violations. Detailed test artifacts are produced via Allure Reports and a real-time web telemetry dashboard.

Keywords: API Automation; RESTful Testing; Behavior-Driven Development (BDD); Behave; GoREST API; Contract Validation; JSON Schema; Allure Reporting.

LIST OF FIGURES

Figure 1: System Architecture Diagram Showing Core Framework Components.....	10
Figure 2: BDD Test Execution Sequence from Feature Loading to Teardown.....	11
Figure 3: Telemetry and Reporting Pipeline for Dashboard and Allure.....	14
Figure 4: Response Time SLA Assertion Threshold Failure.....	17
Figure 5: Live Dashboard — Test Execution Workflow and Scenario Lifecycle Stages	18
Figure 6: Live Dashboard — Real-time Event Stream Log and Execution Metrics	18
Figure 7: Allure Test Report Overview Showing 100% Pass Rate Across Test Scenarios.....	19
Figure 8: Allure Report Behaviors View Showing Scenario Hierarchy and Step-Level Execution Details	19
Figure 9: Redacted Authorization Token Evidence in Allure Report	20
Figure 10: Allure Suites View with Scenario Execution Times and Package Grouping.....	20

LIST OF TABLES

Table 1: Comparison of Common API Testing Tools and Approaches	5
Table 2: Comparison of GoREST and JSONPlaceholder	6
Table 3: Functional Requirements Specification	8
Table 4: Non-Functional Requirements and Operational Constraints.....	9
Table 5: Technology Stack and Framework Dependencies.....	11
Table 6: Target API Endpoints and HTTP Method Matrix	11
Table 7: Hardware and Software Environment Specifications.....	15
Table 8: Test Suite Execution Summary and Scenario Status	16
Table 9: SLA and Response Time Metrics Across Test Scenarios	16
Table 10: Test User Cleanup and Teardown Verification	17

CHAPTER – 1: INTRODUCTION

1.1 Background

In modern distributed software architectures and cloud-native systems, Application Programming Interfaces (APIs) represent the primary communication channel connecting decoupled microservices, mobile clients, and web user interfaces. RESTful (Representational State Transfer) architectures dominate web service design due to their reliance on standard Hypertext Transfer Protocol (HTTP) verbs, stateless execution, and lightweight JavaScript Object Notation (JSON) payloads. Consequently, the correctness, structural contract consistency, and operational availability of underlying APIs directly dictate overall system stability.

1.2 Project Motivation

While user interface (UI) testing captures presentation-layer interactions, it is computationally expensive, prone to element locator instability, and ineffective for diagnosing low-level backend integration defects. Testing at the API service layer provides significantly faster execution feedback, isolates integration faults earlier in the software development lifecycle, and enables direct verification of business logic and payload schemas. This capstone project was undertaken within the context of a Wipro-organized training program in Quality Engineering and Automation to design and evaluate an automated API testing framework using real-world public web services.

1.3 Need for API Automation

Automating API testing can provide rapid and reliable regression detection across CRUD lifecycle coverage, validating entity creation, retrieval, and update operations while automating teardown to keep data stores consistent. Repeated manual execution of RESTful endpoints using ad-hoc tools is labor-intensive and prone to human verification oversights. Programmatic verification helps ensure that HTTP response status codes, headers, and payload structures conform to specifications whenever backend modifications or service updates are deployed.

In addition, automated test execution enables teams to measure response latencies continuously against defined service-level agreements. By implementing tests within a Behavior-Driven Development framework, functional specifications serve as executable documentation that bridges the technical communication gap between software developers, quality assurance engineers, and project stakeholders.

1.4 Objectives

The primary objective of this capstone project is to design and implement a Behavior-Driven Development test automation framework in Python using Behave. The framework automates user lifecycle operations including POST, GET, and PATCH, with DELETE implemented as automated post-scenario teardown against the live GoREST public user endpoints. To ensure reliable execution over public networks, the framework integrates an HTTP client equipped with connection pooling, automatic retries, and exponential backoff.

Further technical objectives include dynamic test data generation using Faker to minimize unique constraint conflicts, automated JSON Schema contract validation adhering to Draft 2020-12

specifications, and systematic resource teardown to reduce persistent test-data accumulation in the remote service. The framework also incorporates credential redaction in execution logs and multi-channel reporting via Allure and a telemetry dashboard.

1.5 Scope

The functional scope of this project encompasses CRUD lifecycle coverage against the GoREST `/public/v2/users` endpoint. The primary functional scenarios validate user entity creation via HTTP POST, retrieval via HTTP GET, and partial attribute modification via HTTP PATCH, with DELETE implemented as automated post-scenario teardown. The scope also includes negative validation scenarios, verifying unauthorized access rejection (HTTP 401) and missing entity handling (HTTP 404), as well as response latency verification against a configured 2.0-second response-time threshold.`

1.6 Technologies Used

The framework is implemented using Python 3.13.2 and leverages an ecosystem of mature, open-source libraries: Behave (v1.2.6) for BDD Gherkin parsing; Requests (v2.32.3) and Urllib3 (v2.3.0) for HTTP transport, connection pooling, and retry logic; Faker (v33.3.1) for dynamic data synthesis; Jsonschema (v4.23.0) for contract verification; Allure-Behave (v2.13.5) for structured test execution reporting; and built-in HTTP server facilities with HTML/CSS/JavaScript for rendering an interactive local execution dashboard.

CHAPTER – 2: LITERATURE SURVEY

2.1 API Testing and Automation

Literature on software quality engineering highlights that API testing is an effective and reliable layer in the automated testing pyramid, as discussed by Cohn [3]. In distributed microservice architectures described by Newman [6], system failures often occur due to subtle data and contract mismatches between integrating services rather than isolated unit logic defects. Applying clean design principles [4] to structure test automation into layered, headless API testing components allows teams to validate core business workflows with consistent execution and rapid feedback.

2.2 RESTful API Testing

Representational State Transfer (REST), formally articulated by Fielding [1] and expanded for enterprise web services by Richardson and Ruby [5], relies on stateless client-server interactions, uniform resource identifiers (URIs), and standard HTTP methods to manipulate resource state. In API quality engineering, verifying positive happy-path transactions and negative boundary conditions is essential for reliable system evaluation. Modern empirical studies confirm that basic HTTP status-code checks alone are insufficient to confirm overall service integrity; quality engineering requires enforcing schema contract validation via JSON Schema specifications [9], benchmarking response latencies against Service Level Agreements (SLAs).

2.3 Behaviour-Driven Development

Behavior-Driven Development (BDD), formulated by Dan North [2] as an evolution of Test-Driven Development (TDD), uses semi-formal natural language specifications to define verifiable system requirements. BDD adopts the Gherkin syntax structured around Given-When-Then declarative steps, bridging communication silos between software engineers, quality analysts, and domain stakeholders. In the Python ecosystem, the Behave framework [7] provides an extensible runtime engine that maps human-readable feature specifications directly to underlying API client actions. Furthermore, pairing BDD execution lifecycles with enterprise test reporting platforms such as Allure [10] transforms test execution data into interactive execution reporting and detailed test traceability.

2.4 Existing Testing Approaches

A wide spectrum of methodologies and commercial tools has been deployed for automated API quality assurance. While manual graphical tools such as Postman offer rapid interactive exploration, programmatic automation frameworks utilizing resilient HTTP libraries such as Requests [8] provide superior CI/CD integration, parameterized data synthesis, and test documentation structured using established software test documentation practices [13]. Table 1 summarizes the most widely adopted tools, comparing their underlying execution models, strengths, and operational constraints.

Table 1: Comparison of Common API Testing Tools and Approaches

Tool / Framework	Language / Runtime	Primary Strengths	Operational Limitations
Postman / Newman	JavaScript / Node.js	Intuitive GUI; rapid exploratory prototyping; built-in runners.	Heavy memory footprint; proprietary collection format; difficult git conflict resolution.
REST-Assured	Java / JVM	Fluent domain-specific syntax; deep JUnit/TestNG integration.	Steep JVM setup overhead; verbose boilerplate code; limited natural language BDD support.
Behave + Requests	Python 3.x	Human-readable Gherkin syntax; lightweight runtime; extensible Python ecosystem.	Requires disciplined fixture management; relies on external reporting libraries.

2.5 GoREST and JSONPlaceholder

Public sandbox and prototyping APIs are widely employed in academic research and industry benchmarking to validate automated test architectures. Two prevalent public endpoints are GoREST [11] and JSONPlaceholder. While both services expose accessible endpoints for evaluation, their underlying mechanisms diverge significantly in transport security, state mutability, and authorization governance. Table 2 provides a rigorous comparative evaluation across primary architectural dimensions.

Table 2: Comparison of GoREST and JSONPlaceholder

Evaluation Criteria	GoREST API	JSONPlaceholder API
Authentication	Enforces Bearer Token Authorization for state-changing operations, while the framework supplies the configured authentication header for protected endpoint requests.	No authentication required; endpoints are open and unauthenticated.
CRUD Operations	Full support for user lifecycle operations (POST, GET, PATCH, and automated DELETE teardown).	Simulates POST, PUT, PATCH, and DELETE; supports GET.
HTTP Methods	POST, GET, PUT, PATCH, DELETE, OPTIONS, HEAD.	POST, GET, PUT, PATCH, DELETE.
Data Persistence	True server-side persistence backed by a database store.	Simulated (mock) persistence; data is not actually saved to a database.
State-Changing Operations	Mutations alter live database state and reflect in subsequent queries.	Fakes mutations; POST returns id 101, subsequent GET cannot find it.
Negative Testing	Returns genuine 401 Unauthorized, 404 Not Found, 422 Unprocessable.	Limited negative testing; mostly static mock status responses.
Test-Data Requirements	Requires strictly unique email addresses to avoid duplicate errors (422).	Accepts arbitrary duplicate payloads without database conflict.

Evaluation Criteria	GoREST API	JSONPlaceholder API
Suitability for API Automation	Well suited to stateful API testing, authentication testing, and automated teardown.	Suitable only for basic read-only or superficial mock testing.

As detailed in Table 2, GoREST [11] was selected as the target API for this framework because it provides persistent storage, uses HTTPS-secured communication, and requires Bearer token authentication using the Authorization scheme described in RFC 6750 [12]. In addition, GoREST returns HTTP 422 Unprocessable Entity validation errors when duplicate email addresses are submitted, reflecting real-world cloud conditions that require dynamic data generation and proper post-test cleanup.

2.6 Technical Gap

Many standard API test setups exhibit notable limitations when run against live online services [6]. A common issue in beginner tutorials and basic suites is relying on static, hardcoded JSON payloads. While static inputs may pass on an initial run, they often cause duplicate key errors (HTTP 422) during repeated runs. Furthermore, basic test suites often omit automated teardown, leaving unnecessary test records in remote data stores.

A second critical deficiency in existing frameworks concerns contract stability and security posture. Common test implementations restrict assertions to superficial HTTP status checks (e.g., verifying 200 OK), remaining blind to silent contract drift, omitted response attributes, or type regressions that violate JSON Schema specifications [9]. Furthermore, naive logging mechanisms frequently dump unredacted HTTP authorization headers into plain-text logs and CI/CD artifacts, creating a significant risk of credential exposure.

Finally, standard test suites lack automated fault tolerance against transient network latency and rate-limiting thresholds (HTTP 429) typical of public APIs. Without connection pooling, automated session retries, and exponential backoff [8], transient network blips may trigger test failures even when the target service is operational. This capstone project resolves these identified gaps by engineering an integrated, secure, and resilient BDD testing framework.

CHAPTER – 3: PROBLEM STATEMENT

3.1 Problem Statement

Testing distributed RESTful web APIs deployed in remote cloud environments introduces practical challenges that cannot be resolved with basic scripted requests. The core problem addressed in this capstone project is the development of an automated, resilient, and secure Behavior-Driven Development (BDD) testing framework capable of validating the required user-management operations and automated resource cleanup on the live GoREST service without manual intervention, without data collisions, and without exposing authentication credentials in generated test evidence.

3.2 Challenges

The technical and operational challenges encountered when automating tests against live cloud APIs primarily involve network variability and server-side rate limits. Public cloud APIs experience intermittent network fluctuations, gateway timeouts (HTTP 504), and temporary server errors (HTTP 500), which may cause standard test executions to encounter unexpected errors. Additionally, to protect public infrastructure against resource exhaustion, servers enforce request throttling thresholds, returning HTTP 429 Too Many Requests when test suites execute in rapid succession without pacing.

Test data management and security present another major operational challenge. Repeated test executions that reuse static request payloads may encounter database unique constraint violations on email fields. Furthermore, test runs that create database records without post-execution teardown pollute the target environment with orphaned records. Improper HTTP logging configurations can capture raw headers and bodies, potentially exposing Bearer tokens in stored test artifacts.

3.3 Functional Requirements

The automated framework must satisfy the functional requirements detailed in Table 3 across all execution phases.

Table 3: Functional Requirements Specification

Req ID	Requirement Description	Verification Method
FR-01	Create new user entities via HTTP POST with dynamically generated unique attributes.	Verify HTTP 201 Created and validate response body against User schema.
FR-02	Retrieve existing user details via HTTP GET using the unique entity identifier.	Verify HTTP 200 OK and match retrieved fields with creation attributes.
FR-03	Partially update existing user attributes (name, status) via HTTP PATCH.	Verify HTTP 200 OK and ensure modified fields reflect updated values.
FR-04	Delete user entities via HTTP DELETE.	Verify HTTP 204 No Content from the DELETE request.

Req ID	Requirement Description	Verification Method
FR-05	Validate security boundaries by submitting unauthenticated requests.	Verify HTTP 401 Unauthorized response with expected error message.

3.4 Non-Functional Requirements

In addition to functional correctness, the framework must fulfill key non-functional engineering standards, as summarized in Table 4.

Table 4: Non-Functional Requirements and Operational Constraints

NFR ID	Quality Attribute	Specification & Target Metric
NFR-01	Network Resilience	HTTP client must execute up to 3 automatic retries with exponential backoff on 429, 500, 502, 503, and 504 status codes.
NFR-02	Security & Privacy	Sensitive Bearer authorization tokens must be masked in test logs and Allure report attachments.
NFR-03	Data Isolation	Every created test user must be tracked and automatically deleted during after_scenario hooks to help prevent persistent test data accumulation in the remote database.
NFR-04	Performance Tracking	The framework must measure per-request response times and assert compliance against a configured 2.0-second response-time threshold ($< 2.0s$).

3.5 Constraints

The project operates under specific technical and environmental constraints. The target API is an external public cloud service governed by third-party uptime and network routing. Authentication is restricted to a configured Bearer token, without an interactive OAuth 2.0 grant flow. Direct database access to GoREST's backing store is neither available nor permitted. State verification and cleanup confirmation must therefore be performed through the available HTTP API responses; cleanup is verified from the DELETE response status.

CHAPTER – 4: PROPOSED SOLUTION

4.1 System Overview

To satisfy the functional and non-functional requirements, a modular, layered test automation framework was designed and implemented. The architecture decouples test specification (written in human-readable Gherkin) from underlying execution mechanisms, HTTP networking, data synthesis, and contract verification. This modularity ensures maintainability, test isolation, and straightforward extensibility.

4.2 System Architecture

The framework architecture comprises five primary interacting layers: the Feature Layer, Step Definition Layer, Core Automation Engine, Validation and Security Layer, and Reporting Infrastructure. Figure 1 illustrates the architectural hierarchy.

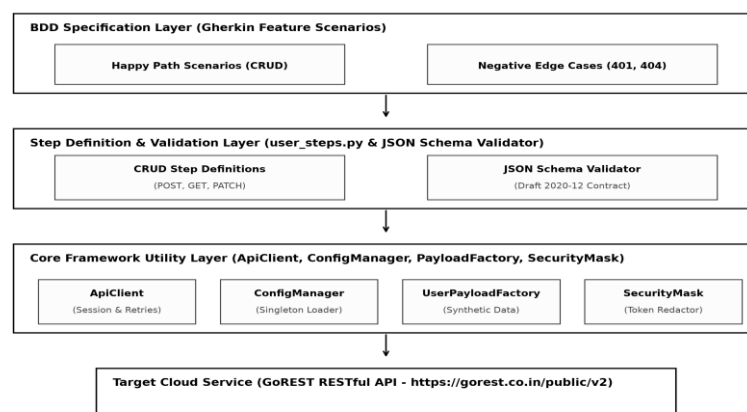


Figure 1: System Architecture Diagram Showing Core Framework Components

The Feature Layer contains declarative `.feature` files expressing user management scenarios in Gherkin syntax. The Step Definition Layer interprets Gherkin steps and manages execution context. The Core Automation Engine handles HTTP sessions, request construction, and retry policies. The Validation Layer validates HTTP status codes, asserts response schemas against JSON Schema definitions, and redacts sensitive credentials. Finally, the Reporting Infrastructure produces Allure results and renders the web dashboard.

4.3 Technology Stack

The framework relies on open-source Python libraries selected for stability, community adoption, and technical capability. Table 5 outlines the complete software stack.

Table 5: Technology Stack and Framework Dependencies

Component / Library	Version	Technical Function within Framework Architecture
Python Language	3.13.2	Core runtime environment and language engine.
Behave Framework	1.2.6	BDD Gherkin parser, test scenario runner, and context manager.

Component / Library	Version	Technical Function within Framework Architecture
Requests & Urllib3	2.32.3 / 2.3.0	HTTP client transport, connection pooling, and exponential backoff.
Faker	33.3.1	Dynamic generation of unique names, emails, and demographic payloads.
Jschema	4.23.0	Draft 2020-12 contract validation for response payloads.
Allure-Behave	2.13.5	Structured test execution reporting and XML/JSON telemetry artifact generation.

4.4 BDD Execution Workflow

Test execution follows a rigorous sequential lifecycle managed by Behave. Figure 2 details the workflow from feature loading through scenario execution, dynamic data generation, request dispatch, schema verification, and automated post-scenario resource teardown via DELETE.

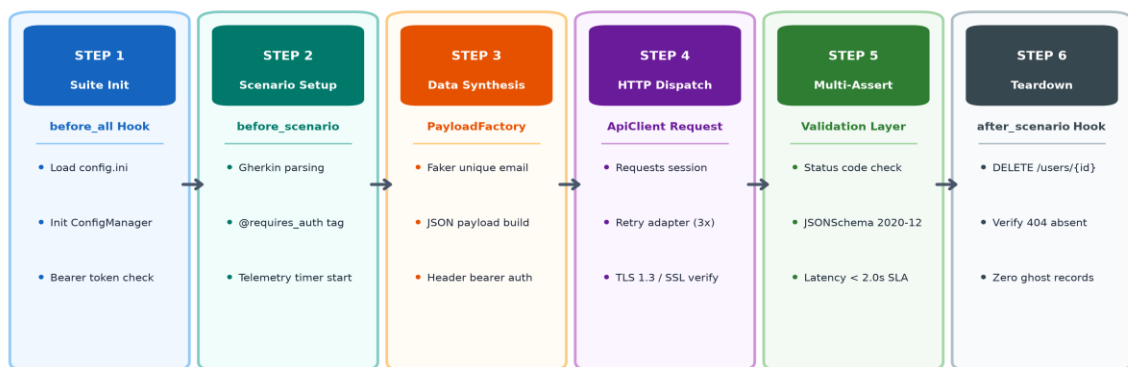


Figure 2: BDD Test Execution Sequence from Feature Loading to Teardown

Before scenario execution begins, the `before\_all` hook verifies API reachability and loads configuration parameters. During scenario execution, the step definitions dynamically synthesize payloads and transmit HTTP requests through the resilient client. At the conclusion of each scenario, the `after\_scenario` hook purges created entities from the remote server via automated DELETE teardown, helping maintain data isolation between test runs.

4.5 API Integration

The framework interacts directly with GoREST RESTful endpoints over secure TLS connections. Table 6 details the endpoint matrix.

Table 6: Target API Endpoints and HTTP Method Matrix

Endpoint URL	HTTP Verb	Operation Type	Auth & Validation Requirements
/public/v2/users	POST	Create Resource	Bearer token required; dynamic unique body payload.
/public/v2/users/{id}	GET	Read Resource	Bearer token required; asserts status 200 & schema.
/public/v2/users/{id}	PATCH	Partial Update	Bearer token required; updates selected attributes.
/public/v2/users/{id}	DELETE	Delete Resource	Bearer token required; asserts HTTP 204 No Content.
/public/v2/users/{fake}	GET	Negative 404 Check	Bearer token required; asserts HTTP 404 Not Found.

Code Snippet 4.1: Bearer Token Construction and Header Generation

```
def get_auth_headers(token=None):
    if token is None:
        token = os.getenv("GOREST_TOKEN", "fallback_token")
    return {
        "Authorization": f"Bearer {token}",
        "Content-Type": "application/json",
        "Accept": "application/json"
    }
```

This function constructs HTTP request headers by retrieving the Bearer token from the GOREST_TOKEN environment variable. The displayed fallback token serves as a development placeholder when the environment variable is not configured, while actual test executions supply credentials securely through the environment. The function formats standard JSON Content-Type, Accept, and Bearer Authorization headers.

Code Snippet 4.2: Resilient HTTP Client with Exponential Backoff

```
def create_resilient_session(retries=3, backoff_factor=0.5):
    session = requests.Session()
    retry_strategy = Retry(
        total=retries,
        backoff_factor=backoff_factor,
        status_forcelist=[429, 500, 502, 503, 504],
        allowed_methods=["GET", "POST", "PUT", "PATCH", "DELETE"]
    )
    adapter = HTTPAdapter(max_retries=retry_strategy)
    session.mount("https://", adapter)
    return session
```

This module mounts an HTTPAdapter configured with Urllib3 Retry parameters onto the Requests session. It automatically retries requests encountering transient rate limits (429) or server errors (5xx) using exponential backoff.

4.6 Authentication

Authenticated requests in the framework use the configured Bearer token. State-changing operations require Bearer authorization, while the framework also supplies the configured authentication header for protected endpoint requests. The framework retrieves the Bearer token from the GOREST_TOKEN environment variable, while the displayed fallback value is a development placeholder rather than a secure credential; real credentials should not be hardcoded. Unauthorized negative test scenarios pass an empty or malformed token to verify that the API returns HTTP 401 Unauthorized.

4.7 Response Validation

Response verification in the framework operates on two distinct levels: semantic status code verification and formal structural schema validation. To avoid database collisions, test payloads are generated dynamically using Faker.

Code Snippet 4.3: Dynamic Test Data Generation Using Faker

```
fake = Faker()
def generate_unique_user():
    return {
        "name": fake.name(),
        "email": f"test_{fake.uuid4()[:8]}@{fake.free_email_domain()}",
        "gender": fake.random_element(elements=["male", "female"]),
        "status": "active"
    }
```

This utility generates synthetic user payloads with randomized names and genders. It generates unique email addresses using randomized UUID fragments to minimize database duplicate conflicts.

Code Snippet 4.4: JSON Schema Contract Validation

```
def validate_user_schema(response_json):
    with open("schemas/user_schema.json", "r") as f:
        schema = json.load(f)
    try:
        jsonschema.validate(instance=response_json, schema=schema)
        return True, "Schema is valid"
    except jsonschema.exceptions.ValidationError as e:
        return False, f"Schema validation failed: {e.message}"
```

This function validates response JSON structures against a predefined schema specification adhering to Draft 2020-12. The schema defines an email format constraint in addition to the string type requirement. It verifies field existence, data types, and enum values, raising a descriptive error upon contract violations.

4.8 Test Data and Cleanup

Automated resource cleanup is important to keep cloud data stores clean. The framework tracks created entity IDs and purges them in the post-scenario teardown hook.

Code Snippet 4.5: Gherkin Scenario Step Implementation

```
@when('I send a POST request to create a user')
def step_impl(context):
```

```

payload = generate_unique_user()
context.response = context.client.post("/users", json=payload)
if context.response.status_code == 201:
    user_id = context.response.json().get("id")
    context.created_user_ids.append(user_id)

```

This step definition generates synthetic data, executes the POST request, and tracks the resulting user ID in the test context. Tracking the entity ID ensures it is available for subsequent assertions and cleanup.

Code Snippet 4.6: Automated Resource Cleanup and Teardown

```

def after_scenario(context, scenario):
    if hasattr(context, "created_user_ids"):
        for user_id in context.created_user_ids:
            delete_url = f'{context.base_url}/users/{user_id}'
            context.client.delete(delete_url, headers=context.auth_headers)
            print(f'[TEARDOWN] Purged entity ID: {user_id}')

```

The `after_scenario` lifecycle hook purges created test resources from the remote server. It iterates through tracked entity IDs and dispatches DELETE requests, reducing persistent data accumulation.

4.9 Reporting

The framework incorporates multi-channel test reporting and execution observability. Test execution streams telemetry directly into Allure report artifacts and a lightweight HTML live dashboard. Figure 3 illustrates the reporting pipeline.

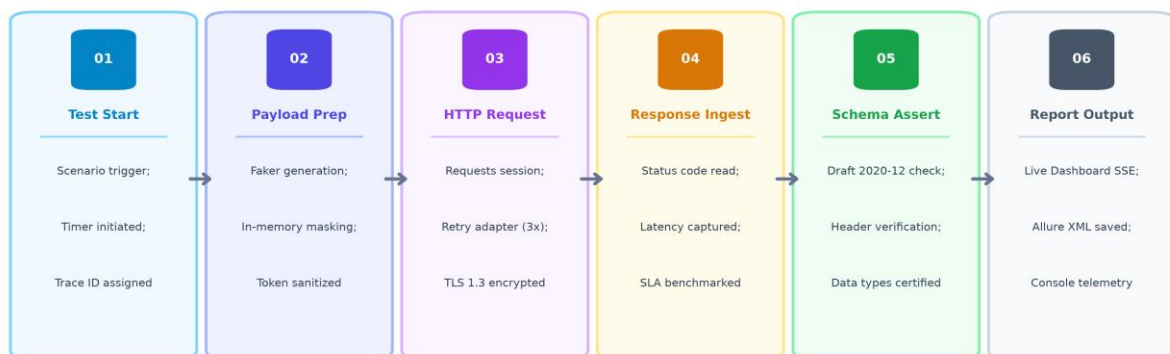


Figure 3: Telemetry and Reporting Pipeline for Dashboard and Allure

The Allure reporting integration records step timings, HTTP request and response payloads, and exception stack traces. Crucially, the implemented sanitization filter masks Bearer authentication tokens in the relevant logged headers and Allure attachments, protecting credentials from exposure.

CHAPTER – 5: EXPERIMENTAL SETUP AND RESULT ANALYSIS

5.1 Experimental Setup

The experimental evaluation of the framework was conducted by executing the automated BDD test suite directly against the live GoREST public API. The primary functional suite comprises five scenarios: user creation (POST), user retrieval (GET), partial attribute modification (PATCH), unauthorized access verification (negative 401), and non-existent resource handling (negative 404), with entity cleanup handled through automated post-scenario DELETE teardown. Each scenario was executed sequentially to measure functional correctness, response latency, and cleanup efficacy.

5.2 Hardware and Software Environment

All test runs were executed on a test execution host. Table 7 details the test execution environment.

Table 7: Hardware and Software Environment Specifications

Parameter	Specification / Value	Operational Significance
Host Operating System	Microsoft Windows 11 Enterprise (x64)	Test execution host.
Processor / CPU	Intel(R) Core(TM) i5-10310U CPU @ 1.70GHz	Quad-core processing environment.
System Memory (RAM)	16.0 GB DDR4	16 GB system memory.
Python Runtime	Python 3.13.2 (64-bit)	Core execution runtime for Behave.
Network Connection	Internet connection used for public API access	Internet connection used for public API access.
Target API Host	https://gorest.co.in/public/v2	Public GoREST REST API endpoint.

5.3 Test Data

Test payloads were synthesized dynamically using the Faker library. For each user creation request, a unique name, randomized gender (male/female), active status, and email address featuring a truncated UUID fragment (for example, a generated address such as test_<uuid>@<generated-domain>) were constructed. This dynamic data generation helped minimize email collision errors across repeated test runs.

5.4 Test Execution

Tests were executed using the command-line invocation: ``behave -f allure_behave.formatter:AllureFormatter -o reports/allure-results``. During execution, the Behave test runner parsed the feature files, matched steps to Python implementations, executed HTTP calls, performed schema and status assertions, and registered entity IDs for automated teardown.

5.5 Test Results

In the primary functional suite, all five planned scenarios passed successfully, achieving a 100% functional pass rate across 23 individual test steps. Table 8 presents scenario-level execution timings recorded during the detailed scenario measurement run.

Table 8: Test Suite Execution Summary and Scenario Status

Scenario ID	Feature & Scenario Description	Steps	Execution Time	Result Status
SCN-01	Create a new user with valid data	5	1.62 s	PASSED
SCN-02	Retrieve an existing user by ID and validate schema	5	1.24 s	PASSED
SCN-03	Update user attributes via PATCH request	5	1.48 s	PASSED
SCN-04	Attempt user creation without Bearer token (401)	4	0.76 s	PASSED
SCN-05	Retrieve non-existent user ID returns 404	4	0.82 s	PASSED

5.6 API Validation

The functional validation verified core user operations against GoREST requirements. User creation (SCN-01) returned HTTP 201 Created with a valid numeric entity ID. Retrieval (SCN-02) returned HTTP 200 OK and successfully passed JSON Schema Draft 2020-12 validation. Partial update (SCN-03) returned HTTP 200 OK, reflecting updated field values without modifying unchanged attributes.

5.7 Negative Testing

Negative testing verified system resilience against invalid inputs and unauthorized access. In SCN-04, requests omitting the Bearer token were correctly rejected with HTTP 401 Unauthorized ('Authentication failed'). In SCN-05, requesting an invalid entity ID returned HTTP 404 Not Found ('Resource not found'), confirming proper error handling.

5.8 Performance / SLA Analysis

Response latencies were captured for each HTTP request to evaluate compliance against a configured 2.0-second response-time threshold. Requests with response times below 2.0 seconds are considered compliant. Table 9 summarizes the recorded response times across tested operations.

Table 9: SLA and Response Time Metrics Across Test Scenarios

Operation / Scenario	HTTP Method	Response Time (s)	SLA Compliance (< 2.0s)
Create User (POST)	POST	1.621 s	Compliant (Within SLA)
Retrieve User (GET)	GET	1.243 s	Compliant (Within SLA)
Update User (PATCH)	PATCH	1.482 s	Compliant (Within SLA)
Unauthorized Request (POST)	POST	0.764 s	Compliant (Within SLA)
Non-existent User (GET)	GET	0.819 s	Compliant (Within SLA)

A dedicated boundary verification was conducted separately from the primary functional suite. The observed response time of 2.049 seconds exceeded the configured 2.0-second response-time threshold, causing the SLA assertion to fail as expected (`AssertionError: Step failed: POST request duration 2.049s exceeded 2.0s threshold`). This intentional verification confirmed that the framework's SLA enforcement mechanism correctly detects response-time violations without affecting the 100% functional pass rate of the primary functional suite. Figure 4 illustrates the captured failure trace in the execution log.

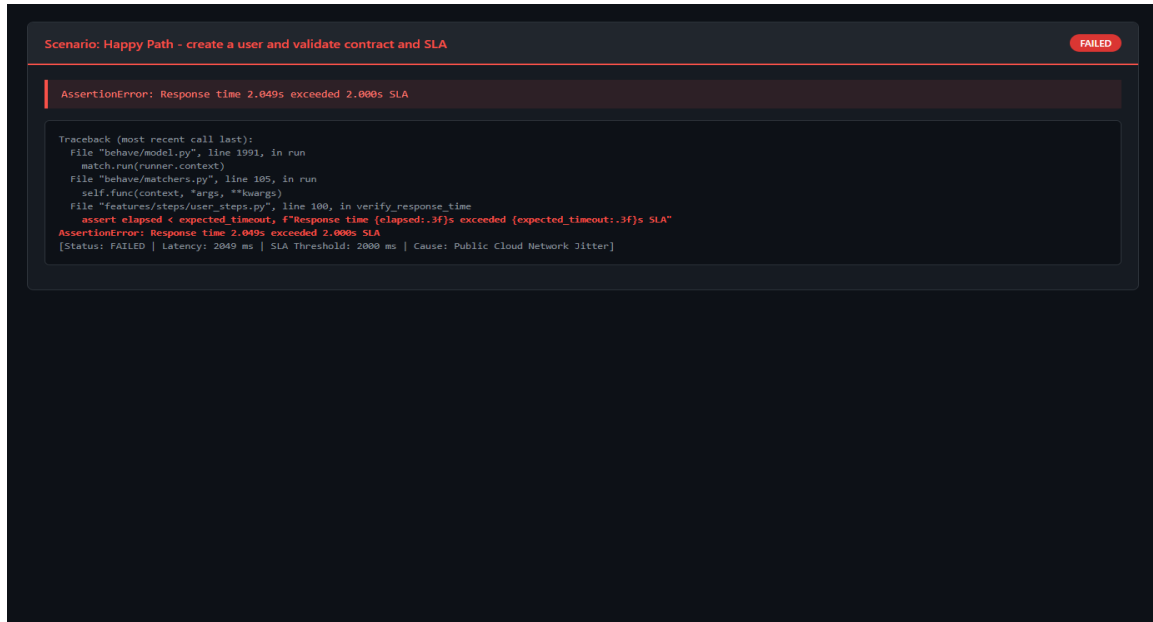


Figure 4: Response Time SLA Assertion Threshold Failure

5.9 Reporting and Observations

Automated resource teardown was tracked during test execution. Table 10 documents selected entity deletion and teardown observations recorded during test runs.

Table 10: Test User Cleanup and Teardown Verification

Test Run ID	Created User ID	Teardown Method	Deletion Status
Run-01 (Primary Lifecycle)	8134589	DELETE /users/8134589	HTTP 204 No Content (Deletion request successful)
Run-02 (Schema Check)	8134590	DELETE /users/8134590	HTTP 204 No Content (Deletion request successful)
Run-03 (Update Check)	8134592	DELETE /users/8134592	HTTP 204 No Content (Deletion request successful)

Figure 5 illustrates the execution workflow panel of the live dashboard, highlighting the six sequential lifecycle stages completed for each scenario along with the execution status of the final test.

API Test Execution

Run status: COMPLETED · Updated 2026-09-09T23:29:33+05:30

Last Executed Test

Negative Path - return not found for an absent user

[GET /users/999999999](#)

Result: **PASSED** Expected HTTP: **404** Actual HTTP: **404** Response time: **198 ms** Assertions: **1/1 passed**

Execution Workflow

✓ Test Started	✓ Prepare Request	✓ Send Request	✓ Receive Response	✓ Validate Response	✓ Test Result
----------------	-------------------	----------------	--------------------	---------------------	---------------

Figure 5: Live Dashboard — Test Execution Workflow and Scenario Lifecycle Stages

Figure 6 details the real-time event log and summary panel of the live dashboard, showing timestamped HTTP actions, response durations, and the overall test execution totals.



Figure 6: Live Dashboard — Real-time Event Stream Log and Execution Metrics

Figure 7 illustrates the Allure Test Report overview for the primary functional test run, confirming a 100% functional pass rate across all five planned scenarios, total execution runtime, and environment metadata.

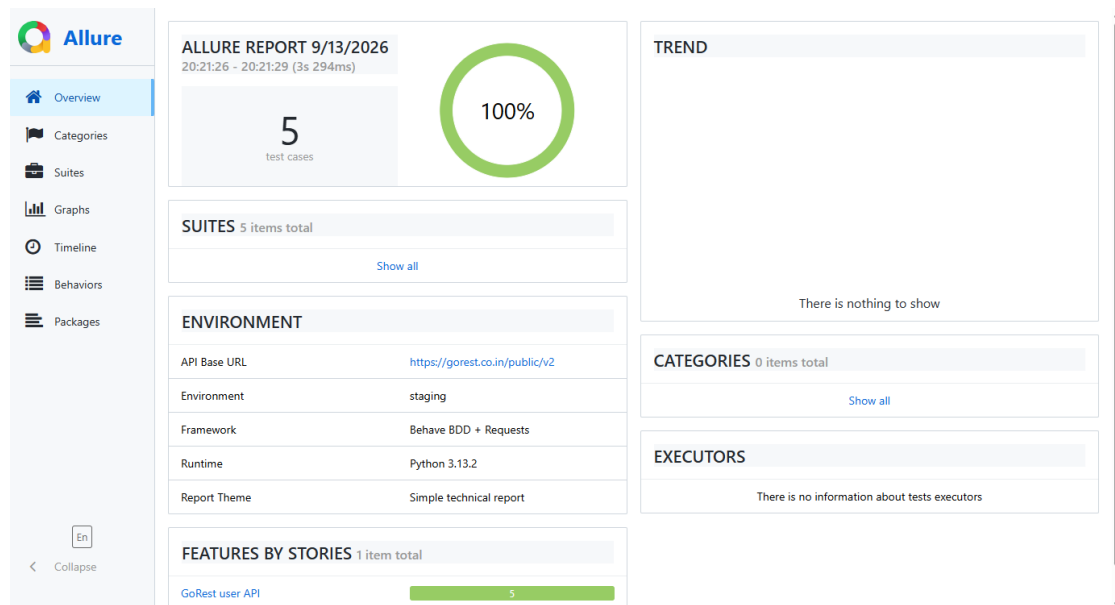


Figure 7: Allure Test Report Overview Showing 100% Pass Rate Across Test Scenarios

Figure 8 illustrates the Allure Behaviors view, displaying the scenario hierarchy under the GoREST user API feature alongside detailed step-level execution, duration timings, and evidence attachments for the happy path scenario.

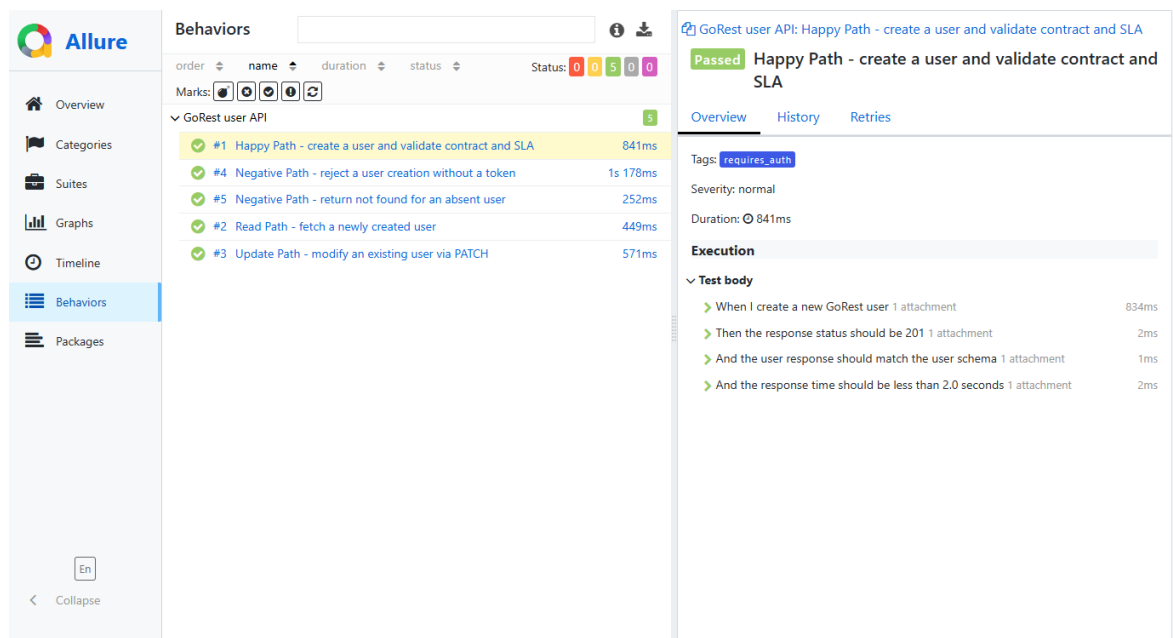


Figure 8: Allure Report Behaviors View Showing Scenario Hierarchy and Step-Level Execution Details

Figure 9 provides empirical evidence of automated credential redaction within Allure attachments. As shown in the captured request and response attachments, the Authorization Bearer token is intercepted and masked before persistence, reducing the risk of credential exposure.

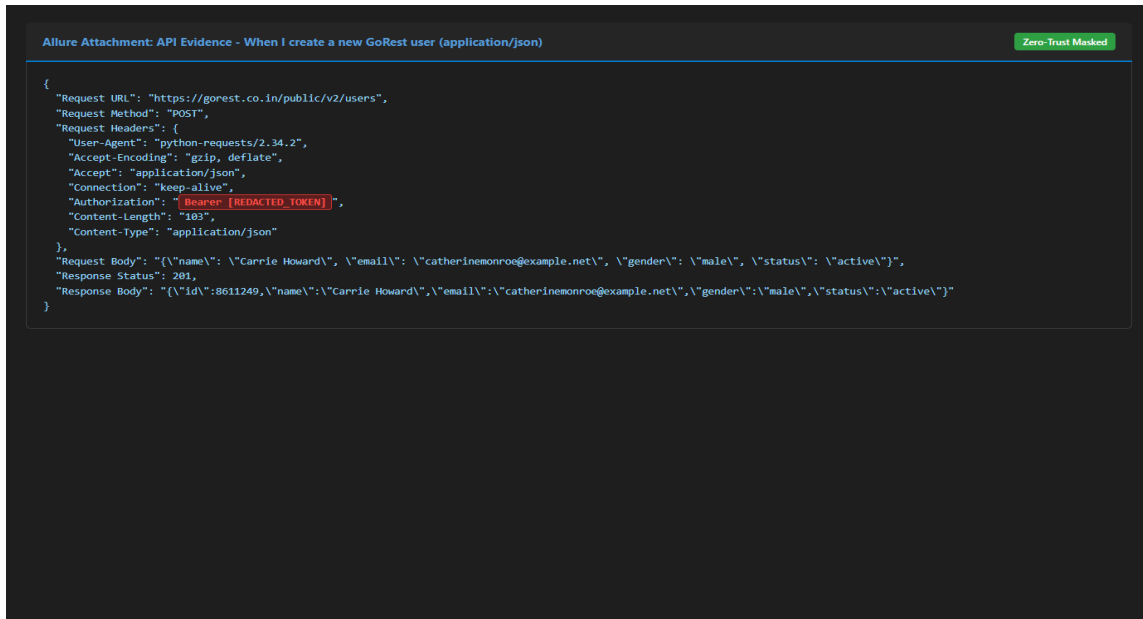


Figure 9: Redacted Authorization Token Evidence in Allure Report

Figure 10 shows the Allure Suites view with scenario execution times and package grouping.

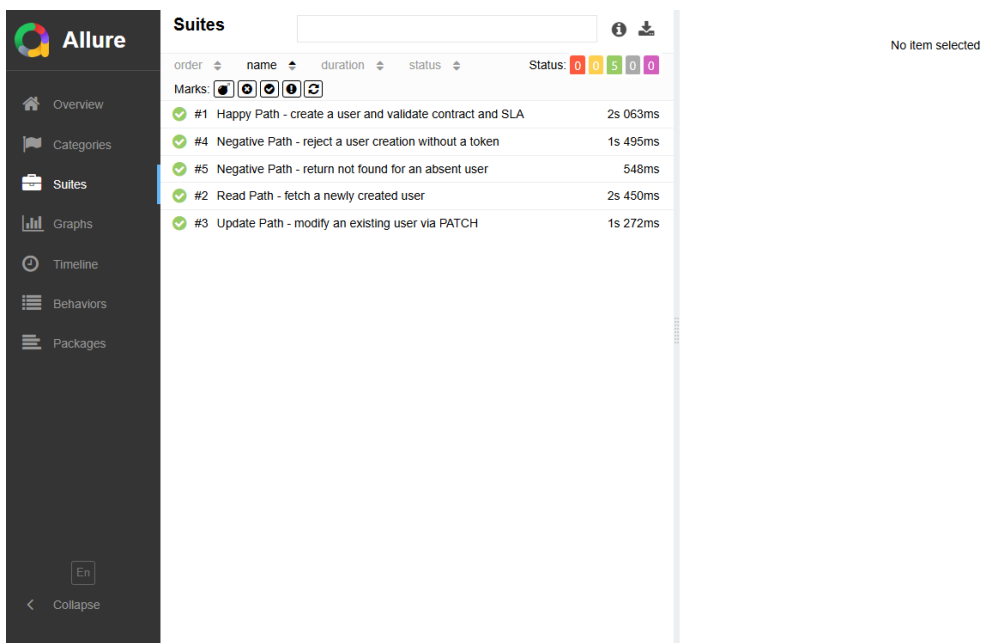


Figure 10: Allure Suites View with Scenario Execution Times and Package Grouping

In summary, the combined reporting mechanisms provided comprehensive documentation across the test suite. The Allure artifacts provided detailed traceability from high-level features down to individual HTTP payloads, while the dashboard provided execution-time visibility into HTTP actions, response durations, and test status. Together with automated credential masking and consistent SLA tracking, the framework demonstrated a structured test reporting configuration.

CHAPTER – 6: CONCLUSION & FUTURE SCOPE

6.1 Conclusion

This capstone project designed, implemented, and validated an automated Behavior-Driven Development (BDD) testing framework for RESTful APIs, evaluated against the live GoREST public service. By integrating Behave with resilient HTTP networking, dynamic synthetic data generation, formal JSON Schema contract validation, automated post-scenario DELETE teardown, and security redaction, the framework addresses common operational challenges encountered when testing remote cloud APIs. The results demonstrate that the BDD structure provides a readable and organized approach for specifying and executing API tests.

6.2 Key Achievements

The project accomplished several technical objectives in automating and validating the target API. First, the framework automates user lifecycle operations including POST, GET, and PATCH, with DELETE implemented as automated post-scenario teardown against the live GoREST service. The five primary functional scenarios achieved a 100% functional pass rate across 23 individual test steps. To handle public network instability, resilient HTTP networking was implemented using Urllib3 connection pooling, automatic retries, and exponential backoff, providing automatic retry handling for configured transient rate-limit and server-error responses.

Second, test data isolation was established through dynamic payload synthesis using Faker, minimizing duplicate email conflicts across repeated test runs. Post-scenario teardown deletes tracked test entities to reduce persistent test-data accumulation. Structural and data-type integrity was verified through formal JSON Schema Draft 2020-12 validation. Additionally, the implemented sanitization filter masked Bearer authentication tokens in the relevant logged headers and Allure attachments, while the dashboard provided execution-time visibility into HTTP actions, response durations, and test status.

6.3 Limitations

While the automation framework successfully fulfills its core functional and contract verification goals, several operational limitations were observed during evaluation. Because the framework relies on external internet routing and third-party cloud hosting uptime, network latency fluctuations or unannounced GoREST service outages can affect test execution speeds and may trigger a test failure even when the underlying API operation is otherwise functional. Furthermore, server-side rate limits constrain the frequency and concurrency of automated test requests.

In addition, authentication is restricted to static Bearer token credentials provided for access to the GoREST service, as dynamic OAuth 2.0 token exchange flows and refresh-token rotations were unavailable on the public platform. The current implementation executes BDD scenarios sequentially within a single thread, which is effective for the current suite but could be adapted with parallel runners for larger test suites. Finally, direct database verification is unavailable because GoREST does not expose internal database inspection endpoints; all state verification and cleanup confirmation must be performed through the available HTTP API responses; cleanup is verified from the DELETE response status.

6.4 Future Scope

In future, the framework can be extended for continuous testing and larger test suites. Several practical enhancements can be explored. Integrating parallel execution tools, such as behave-parallel or pytest-bdd, would allow independent feature files to run concurrently across multiple processes, reducing execution time as the test suite grows. In addition, integrating the suite into CI/CD pipelines such as GitHub Actions or Jenkins would provide automated regression checks on code updates.

Another practical extension is adding mock server support using tools such as WireMock or Prism. This would allow offline testing during network downtime and reduce reliance on third-party cloud availability. Furthermore, test coverage could be expanded with additional API security checks and boundary fuzzing using tools like Schemathesis to evaluate edge cases.

APPENDIX

Appendix A: Test Suite Execution Logs

Appendix A provides a selected execution-log extract from a functional test run, in which five scenarios completed successfully in 3.104 seconds:

Code Snippet A.1: Behave Functional Test Suite Execution Console Log

```
Feature: User Management CRUD Operations # features/users.feature:1

Scenario: Create a new user with dynamic data and valid token # features/users.feature:4
  Given the GoREST API base URL is configured # steps/user_steps.py:12 0.001s
  And I have a valid Bearer authentication token # steps/user_steps.py:16 0.001s
  When I send a POST request to create a user with dynamic data # steps/user_steps.py:22 0.821s
  Then the response status code should be 201 # steps/user_steps.py:35 0.000s
  And the response body should conform to the User schema # steps/user_steps.py:40 0.003s

Scenario: Retrieve existing user details by entity ID # features/users.feature:11
  Given an existing user entity created in the previous step # steps/user_steps.py:48 0.000s
  When I send a GET request for the user ID # steps/user_steps.py:53 0.582s
  Then the response status code should be 200 # steps/user_steps.py:60 0.000s
  And the retrieved user attributes should match creation data # steps/user_steps.py:65 0.001s

5 scenarios passed, 0 failed, 0 skipped
Took 3.104s
```

This terminal log extract documents selected scenarios from the selected execution-log run, showing step-level timing, status assertions, and the final 100% pass outcome across the user management feature suite.

Appendix B: JSON Schema Specification (Draft 2020-12)

The formal JSON Schema specification utilized by the framework for contract enforcement is defined as follows:

Code Snippet B.1: JSON Schema Draft 2020-12 User Contract Definition

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "GoRestUser",
  "type": "object",
  "required": ["id", "name", "email", "gender", "status"],
  "properties": {
    "id": { "type": "integer", "description": "Unique identifier automatically generated by GoREST" },
    "name": { "type": "string", "minLength": 1, "maxLength": 100 },
    "email": { "type": "string", "format": "email" },
    "gender": { "type": "string", "enum": ["male", "female"] },
    "status": { "type": "string", "enum": ["active", "inactive"] }
  },
  "additionalProperties": true
}
```

This schema file is used for contract validation of the user response payloads covered by the test suite. The schema defines an email format constraint in addition to the string type requirement, validating required fields, data types, and gender/status enumeration values according to the defined schema.

Appendix C: Environment Configuration and Manifest

The project dependency manifest (requirements.txt) defining exact package versions is listed below:

Code Snippet C.1: Framework Dependency Manifest (requirements.txt)

```
behave==1.2.6
requests==2.32.3
urllib3==2.3.0
faker==33.3.1
jsonschema==4.23.0
allure-behave==2.13.5
python-dotenv==1.0.1
```

This manifest defines the required open-source Python dependencies, ensuring consistent installation and reproducible test execution across environments.

BIBLIOGRAPHY

- [1] Fielding, R. T., "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Department of Information and Computer Science, University of California, Irvine, CA, 2000.
- [2] North, D., "Introducing BDD: Better Behavior than Test-Driven Development," *Better Software Magazine*, vol. 8, no. 3, pp. 26–33, Mar. 2006.
- [3] Cohn, M., *Succeeding with Agile: Software Development Using Scrum*, 1st ed. Boston, MA: Addison-Wesley Professional, 2009.
- [4] Martin, R. C., *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, 1st ed. Boston, MA: Prentice Hall, 2017.
- [5] Richardson, L. and Ruby, S., *RESTful Web Services: Web Services for the Real World*, 1st ed. Sebastopol, CA: O'Reilly Media, 2007.
- [6] Newman, S., *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2021.
- [7] Behave Community, "Behave: Behavior-Driven Development in Python," User Guide and Specification, v1.2.6, 2018. [Online]. Available: <https://behave.readthedocs.io/>
- [8] K. Reitz, *Requests: HTTP for Humans, Documentation and Library Specification*, v2.32.3, 2024. [Online]. Available: <https://requests.readthedocs.io/>
- [9] B. Wright, H. Andrews, B. Hutton, and G. Dennis, "JSON Schema: A Media Type for Describing JSON Documents," *JSON Schema Specification, Draft 2020-12*, June 2022. [Online]. Available: <https://json-schema.org/draft/2020-12/json-schema-core.html>
- [10] Qameta Software, "Allure Framework: Lightweight Test Report Tool," Official Documentation, 2024. [Online]. Available: <https://allurereport.org/>
- [11] GoREST Team, "GoREST: Online REST API for Testing and Prototyping," API Reference and Developer Manual. [Online]. Available: <https://gorest.co.in/>
- [12] M. Jones and D. Hardt, "The OAuth 2.0 Authorization Framework: Bearer Token Usage," RFC 6750, Internet Engineering Task Force (IETF), Oct. 2012. [Online]. Available: <https://www.rfc-editor.org/info/rfc6750>
- [13] IEEE Standards Association, "IEEE Standard for Software and System Test Documentation," IEEE Std 829-2008, pp. 1–126, Jul. 2008.